24-10-2026

CO2 -ASSSESMENT TOOL 2

SCENARIO BASED

COURSE NAME: NATURAL LANGUAGE PROCESSING

COURSECODE: DSA0308

NAME: R SAI HASNIKA **REG NO:192424178**

QUESTION 1: Morphological Processing for Search Engine Indexing

1.1 Introduction

Morphological processing is a Natural Language Processing (NLP) technique used to analyze the internal structure of words. It identifies the root word, prefixes, and suffixes to understand how words are formed.

In a search engine indexing system, users may search for words such as analyzing, analysis, and analytical. Although these words are related in meaning, their forms are different. Morphological processing helps identify these relationships and improves the accuracy of search results.

1.2 Problem Statement

A large-scale search engine processes millions of documents containing different forms of related words. If the system treats every word as completely different, it may fail to retrieve relevant documents.

Therefore, a rule-based morphological processing system is required to:

- Identify the root or base form of each word.
- Separate suffixes and other morphological components.
- Differentiate between inflectional and derivational forms.
- Normalize related words into a common representation.
- Generate a structured report for efficient search engine indexing.

1.3 Objectives

1. To analyze the morphological structure of the words “analyzing,” “analysis,” and “analytical.”
2. To identify the root word and corresponding suffixes.
3. To classify each word as an inflectional or derivational form.

4. To normalize related word forms for indexing.
5. To improve search accuracy by grouping semantically related terms.

1.4 Morphological Analysis

Morphological analysis divides a word into meaningful components, including the root, prefix, and suffix.

- Root: The central part of a word that carries its basic meaning.
- Prefix: A meaningful element added before a root.
- Suffix: A meaningful element added after a root.
- Inflectional morphology: Changes the grammatical form of a word without creating a fundamentally new word.
- Derivational morphology: Creates a related word that may belong to a different grammatical category or have a modified meaning.
- Normalization: Converts related word forms into a consistent representation for indexing.

1.5 Analysis of the Given Words

Original word	**Root/base**	**Affix or**	**Transformation**	**Normalized**
		structure	type	output
Analyzing	Analyze	-ing	Inflectional	ANALYZE
Analysis	Analysis	Irregular related form of analyze	Derivational relationship	ANALYZE
Analytical	Analyt-	-ical	Derivational	ANALYZE

Note: These words do not all follow the same simple suffix-removal rule. “Analysis” and “analytical” have related forms but require lexical or morphological rules to connect them to “analyze.” The normalized output ANALYZE is a chosen indexing representation rather than the literal root of every word.

1.6 Algorithm

1. Start the program.
2. Store the input words in a list.
3. Define morphological rules for each word.
4. Identify the root or base form.
5. Extract the corresponding suffix or structural relationship.
6. Classify the transformation as inflectional or derivational.
7. Normalize related words to the common indexing form analyze.
8. Display the results in a structured table.
9. Stop the program.

1.7 Python Program

CO2AT21.PY > ...

```
    morphology = {
        "analyzing": {
            "root": "analyze",
            "affix": "-ing",
            "type": "Inflectional",
            "normalized": "analyze"
        },
        "analysis": {
            "root": "analysis",
            "affix": "Irregular related form",
            "type": "Derivational relationship",
            "normalized": "analyze"
        },
        "analytical": {
            "root": "analyt",
            "affix": "-ical",
            "type": "Derivational",
            "normalized": "analyze"
        }
    }
    print("=" * 92)
    print("      MORPHOLOGICAL PROCESSING FOR SEARCH ENGINE INDEXING")
    print("=" * 92)
    print("{:<15} {:<12} {:<24} {:<27} {:<12}".format(
        "Word", "Root/Base", "Affix/Structure",
        "Transformation", "Normalized"
    ))
    print("-" * 92)
    for word in words:
        data = morphology[word]
        print("{:<15} {:<12} {:<24} {:<27} {:<12}".format(
            word,
            data["root"],
            data["affix"],
            data["type"],
            data["normalized"]
        ))
```

1.8 Expected Output

```
================================================================================
      MORPHOLOGICAL PROCESSING FOR SEARCH ENGINE INDEXING
================================================================================
Word            Root/Base     Affix/Structure          Transformation              Normalized
--------------------------------------------------------------------------------
analyzing       analyze       -ing                     Inflectional                analyze
analysis        analysis      Irregular related form   Derivational relationship   analyze
analytical      analyt        -ical                    Derivational                analyze
================================================================================
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py>
```

1.9 Explanation of the Program

1. Input word list: The list stores the three words that must be analyzed.
2. Morphological dictionary: A dictionary stores the root, affix, transformation type, and normalized output for each word.
3. Root identification: The program identifies the base form or related lexical stem according to predefined rules.
4. Affix identification: It records the suffix or structural relationship associated with each word.
5. Transformation classification: It distinguishes grammatical changes from derivational word formation.
6. Normalization: All three words are mapped to analyze so the indexing system can group them under one selected representation.
7. Structured display: The format() method aligns the columns and makes the report easy to read.
8. Loop processing: The for loop processes every word without repeating the same printing statements.

1.10 Advantages

- Improves search engine retrieval accuracy.
- Groups related words under a common indexing representation.
- Reduces unnecessary duplication in the search index.
- Supports efficient document searching.
- Helps users retrieve relevant documents even when their search terms differ from the words in those documents.
- Provides a simple and explainable rule-based approach.

1.11 Applications

- Search engine indexing.
- Document retrieval systems.
- Digital libraries.
- Text mining.
- Information retrieval and NLP applications.

1.12 Conclusion

Morphological processing helps search engines understand the relationships between different forms of words. By identifying roots, affixes, and transformation types, the proposed rule-based program maps “analyzing,” “analysis,” and “analytical” to a common indexing representation. This improves the ability to retrieve documents containing related terminology. In a real-world system, a linguistic lexicon or an advanced morphological analyzer would be required to handle irregular word relationships accurately.

QUESTION:2Morphological Parsing for AI-Based Sentiment Analysis

2.1 Introduction

Morphological parsing is an NLP technique used to identify the internal structure of words and understand how prefixes and suffixes affect their meanings.

In sentiment analysis, product reviews may contain words such as “disagree,” “agreement,” and “agreeable.” These words share the base concept of agree, but their meanings differ because of the addition of prefixes and suffixes.

A morphological parser helps an AI system identify these differences and interpret the words more consistently.

2.2 Problem Statement

An AI-based sentiment analysis platform processes customer reviews containing different forms of the same base word. If the system treats every form as unrelated, it may fail to understand the relationship between words or interpret their sentiment incorrectly.

Therefore, a morphological parsing system is required to:

- Detect prefixes and suffixes.
- Identify the root or base word.
- Classify morphological transformations.
- Explain how affixes change meaning.
- Normalize related forms for further analysis.
- Generate a structured report for sentiment analysis.

2.3 Objectives

1. To analyze “disagree,” “agreement,” and “agreeable.”
2. To identify prefixes, roots, and suffixes.
3. To classify the transformations as derivational or inflectional.
4. To understand how affixes modify the meaning of the base word.
5. To normalize related forms to agree.
6. To support consistent language processing in AI-based sentiment analysis.

2.4 Morphological Parsing Concepts

- Prefix: A word element added before the base word. For example, dis- in “disagree.”

- Root/base word: The central word that carries the main meaning. Here, the common base is agree.
- Suffix: A word element added after the base word. For example, -ment in “agreement.”
- Derivational transformation: A change that modifies meaning or grammatical category.
- Semantic interpretation: The meaning conveyed by the complete word.
- Normalization: Mapping different word forms to a common base representation.

2.5 Analysis of the Given Words

Word	Prefix	Root	Suffix	Transformation	Semantic interpretation
Disagree	dis-	agree	None	Derivational	To have a different or opposing opinion
Agreement	None	agree	-ment	Derivational	A state or result of agreeing
Agreeable	None	agree	-able	Derivational	Pleasant, acceptable, or willing to agree

All three words can be normalized to agree for base-word indexing. However, normalization should not remove their original meanings when the system calculates sentiment.

2.6 Algorithm

1. Start the program.
2. Store the input words in a list.
3. Define the root word agree.
4. Check each word for recognized prefixes and suffixes.
5. Extract the root and morphological components.
6. Classify each transformation as derivational or inflectional.
7. Describe the semantic meaning of each word.
8. Normalize all words to the common base agree.
9. Display the results in a structured table.
10. Stop the program.

2.7 Python Program

CO2AT21.PY > ...

```python
words = ["disagree", "agreement", "agreeable"]
morphology = {
    "disagree": {
        "prefix": "dis-",
        "root": "agree",
        "suffix": "None",
        "type": "Derivational",
        "meaning": "Opposing or having a different opinion",
        "normalized": "agree"
    },
    "agreement": {
        "prefix": "None",
        "root": "agree",
        "suffix": "-ment",
        "type": "Derivational",
        "meaning": "A state or result of agreeing",
        "normalized": "agree"
    },
    "agreeable": {
        "prefix": "None",
        "root": "agree",
        "suffix": "-able",
        "type": "Derivational",
        "meaning": "Pleasant, acceptable, or willing to agree",
        "normalized": "agree"
    }
}
print("=" * 112)
print("             MORPHOLOGICAL PARSING FOR SENTIMENT ANALYSIS")
print("=" * 112)
print("{:<13} {:<10} {:<10} {:<10} {:<16} {:<37} {:<12}".format(
    "Word", "Prefix", "Root", "Suffix",
    "Transformation", "Semantic Meaning", "Normalized"
))
print("-" * 112)
for word in words:
    data = morphology[word]
    print("{:<13} {:<10} {:<10} {:<10} {:<16} {:<37} {:<12}".format(
        word,
        data["prefix"],
        data["root"],
        data["suffix"],
        data["type"],
        data["meaning"],
        data["normalized"]
    ))

print("=" * 112)
```

2.8

Expected Output

```
============================================================================================================
             MORPHOLOGICAL PARSING FOR SENTIMENT ANALYSIS
============================================================================================================
Word          Prefix     Root       Suffix     Transformation   Semantic Meaning                    Normalized
------------------------------------------------------------------------------------------------------------
disagree      dis-       agree      None       Derivational     Opposing or having a different opinion agree
agreement     None       agree      -ment      Derivational     A state or result of agreeing         agree
agreeable     None       agree      -able      Derivational     Pleasant, acceptable, or willing to agree agree
============================================================================================================
PS C:\Users\Hasnika\OneDrive\Desktop\NLP.py>
```

Normalized base form for all words: agree

Note: The output may wrap depending on the width of the Python terminal.

2.9 Explanation of the Program

1. Input word list: Stores the three words to be analyzed by the program.
2. Morphological dictionary: Contains the prefix, root, suffix, transformation type, meaning, and normalized form for each word.
3. Prefix detection: Identifies dis- in “disagree,” which reverses or negates the basic meaning of agreeing.
4. Suffix detection: Identifies -ment in “agreement,” which forms a noun, and -able in “agreeable,” which forms an adjective.
5. Root extraction: Identifies agree as the common base word.
6. Transformation classification: Classifies all three words as derivational because their affixes modify meaning or grammatical category.
7. Semantic interpretation: Explains the meaning of each complete word to help distinguish the words during sentiment analysis.
8. Normalization: Maps each word to agree for efficient base-word indexing.
9. Formatted output: Uses aligned columns to produce a neat and readable morphological report.

2.10 Role of Morphology in Sentiment Analysis

Morphological analysis supports sentiment analysis by helping the system recognize related words and understand their differences.

For example:

- Disagree: May indicate opposition, dissatisfaction, or a negative opinion depending on the sentence.
- Agreement: May indicate consensus or acceptance, but it is not necessarily positive in every context.
- Agreeable: Often expresses a positive quality, such as being pleasant or acceptable.

Although these words share a common root, they should not automatically receive the same sentiment score. The system must consider the complete word, surrounding words, and review context.

2.11 Advantages

- Identifies relationships between different word forms.
- Improves consistency in text preprocessing.
- Helps distinguish negative prefixes from positive or neutral word forms.
- Supports more meaningful sentiment classification.

- Reduces duplicate representations of related words.
- Provides interpretable output for debugging and evaluation.

2.12 Applications

- Product review analysis.
- Customer feedback classification.
- Social media sentiment analysis.
- Opinion mining.
- Chatbots and conversational AI.
- Customer satisfaction monitoring.

2.13 Conclusion

Morphological parsing helps AI-based sentiment analysis systems identify roots, prefixes, suffixes, and meaning changes in related words. The proposed Python program analyzes “disagree,” “agreement,” and “agreeable,” classifies their transformations, and normalizes them to the base word agree. This provides a useful foundation for text preprocessing. However, accurate sentiment analysis also requires contextual information because morphological relationships alone cannot determine whether a complete review is positive, negative, or neutral.